

B U P T

密码工程

第14讲、IBE与基于国密的典型密码应用 IBE、国密SSL实现

授课教师：王晨宇

时间：2026年6月14日



6.1

IBE介绍

- ✓ 6.1.1 基于身份的加密方案(IBE)
- ✓ 6.1.2 实际的IBE
- ✓ 6.1.3 IBE系统应用实例
- ✓ 6.1.4 IBE的优势和不足



- 1984年, Adi Shamir 首次正式提出 IBE 的概念, 这一设想为密码学领域打开了新的研究方向。
- 1987年, Tanaka基于离散对数问题和大整数的分解问题提出一个修改的IBE实现方案, 并引入了门限的概念来解决该方案不抵抗合谋攻击的问题。
- 第一个实用的IBE方案直到2001年由Boneh等人提出, 该加密方案使用双线性对进行构造, 基于双线性Diffie-Hellman假设, 在随机预言机模型下抵抗适应性选择密文攻击。



- 基于身份的加密(IBE, Identity-Based Encryption)是一种创新的公钥密码体制。
- 与传统的公钥密码体制不同，它允许使用任意字符串，比如用户的姓名、IP地址、电子邮件地址等直接作为公钥。
- 举例说明，在传统加密中，公钥更类似于一串无规律的字符串；而在IBE里，公钥可以是用户的常用邮箱地址，更直观易记。



- 基于身份的加密(IBE)也可以叫做“身份基加密”，设计的基本思想是简化公钥基础设施(PKI)中的密钥管理和分发过程。
- 在传统的PKI中，每个用户都需要一个唯一的公钥和对应的私钥，并且公钥需要通过数字证书来验证其真实性。这涉及到复杂的密钥管理和证书管理过程。
- IBE解决了这个问题，通过将用户的标识符直接用作公钥，任何知道用户标识符的人就可以使用该标识符来加密消息，而不需要获取数字证书。



IBE介绍

6.1.1 IBE架构

B U P T

□ IBE通常由以下部分组成：

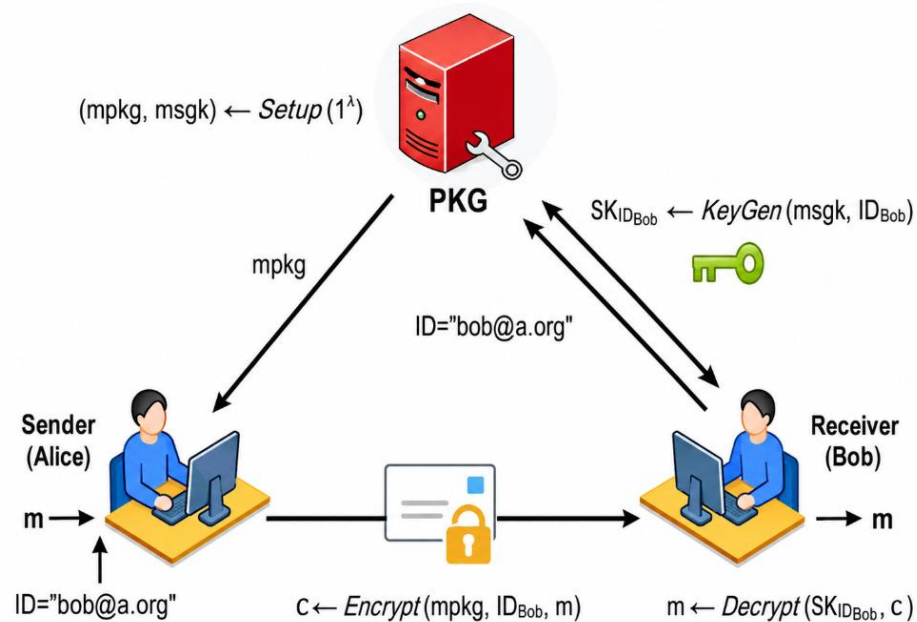
用户、密钥生成中心、公共参数。

□ **用户**：使用IBE进行加密和解密操作。

□ **密钥生成中心 (PKG)**：负责生成和分发私钥材料。

- 密钥生成：PKG生成全局的IBE系统参数和主密钥；
- 密钥派生：当用户注册时，使用主密钥为每个用户生成私钥；
- 密钥分发：PKG安全地分发私钥给相应的用户；
- 密钥撤销：在必要时，PKG可以撤销用户的私钥；

□ **公共参数**：一组公开的数学参数，用于IBE算法的实现。





- 一个IBE方案由四个算法组成：Setup、Encrypt、KeyGen、Decrypt。
- **初始化(Setup)**：初始化算法是一个随机化算法，输入是安全参数 k ，输出为系统参数 $params$ （全程公开）和主密钥 msk 。表示为 $(params, msk) \leftarrow Init(k)$
 - 输入：安全参数 1^k ，其中 k 是安全参数的长度，通常用来控制密钥的大小和计算的安全强度。
 - 输出：全局系统参数 $params$ 和主密钥 msk 。
- $params$ 包含用于IBE操作的所有公共信息，如算法参数、哈希函数等，这些参数对于所有的用户都是相同的，并且是公开的。
- msk 是一个秘密值，用于生成用户的私钥，并且只有密钥生成中心知道它。



- **加密(Encrypt):** 加密算法也是一个随机化算法, 输入是消息 M 、系统参数 $params$ 以及接收方的身份 ID , 输出密文 CT , 仅当接收方具有相同身份 ID 时, 才能解密。表示为 $CT = E_{ID}(M)$
 - 输入: 消息 M 、系统参数 $params$ 以及接收方的身份标识 ID 。
 - 输出: 针对特定接收者 ID 的密文 CT 。
- 这个过程通常涉及使用 ID 和 $params$ 来生成一个公钥, 然后使用这个公钥来加密消息 M 。



□ **密钥生成(KeyGen):** 密钥生成算法也是一个随机化算法, 输入是系统参数 params , 接收方的身份ID以及主密钥 msk , 输出会话密钥 sk 。表示为 $\text{sk} \leftarrow \text{IBEGen}(\text{ID})$

- 输入: 系统参数 params 、接收方的身份标识 ID 以及主密钥 msk 。
- 输出: 与身份标识 ID 相关联的私钥 sk 。

□ 这个私钥由 PKG 使用 msk 生成, 并且只对持有相应身份标识的用户有效。



- **解密(Decrypt):** 解密算法是确定性算法, 输入会话密钥 sk 及密文 CT , 输出消息 M , 表示为 $M = D_{sk}(CT)$
 - 输入: 会话密钥 sk 及密文 CT 。
 - 输出: 原始消息 M 。

- 只有持有与密文 CT 中指定身份标识相匹配的私钥 sk 的用户能解密出原始消息 M 。

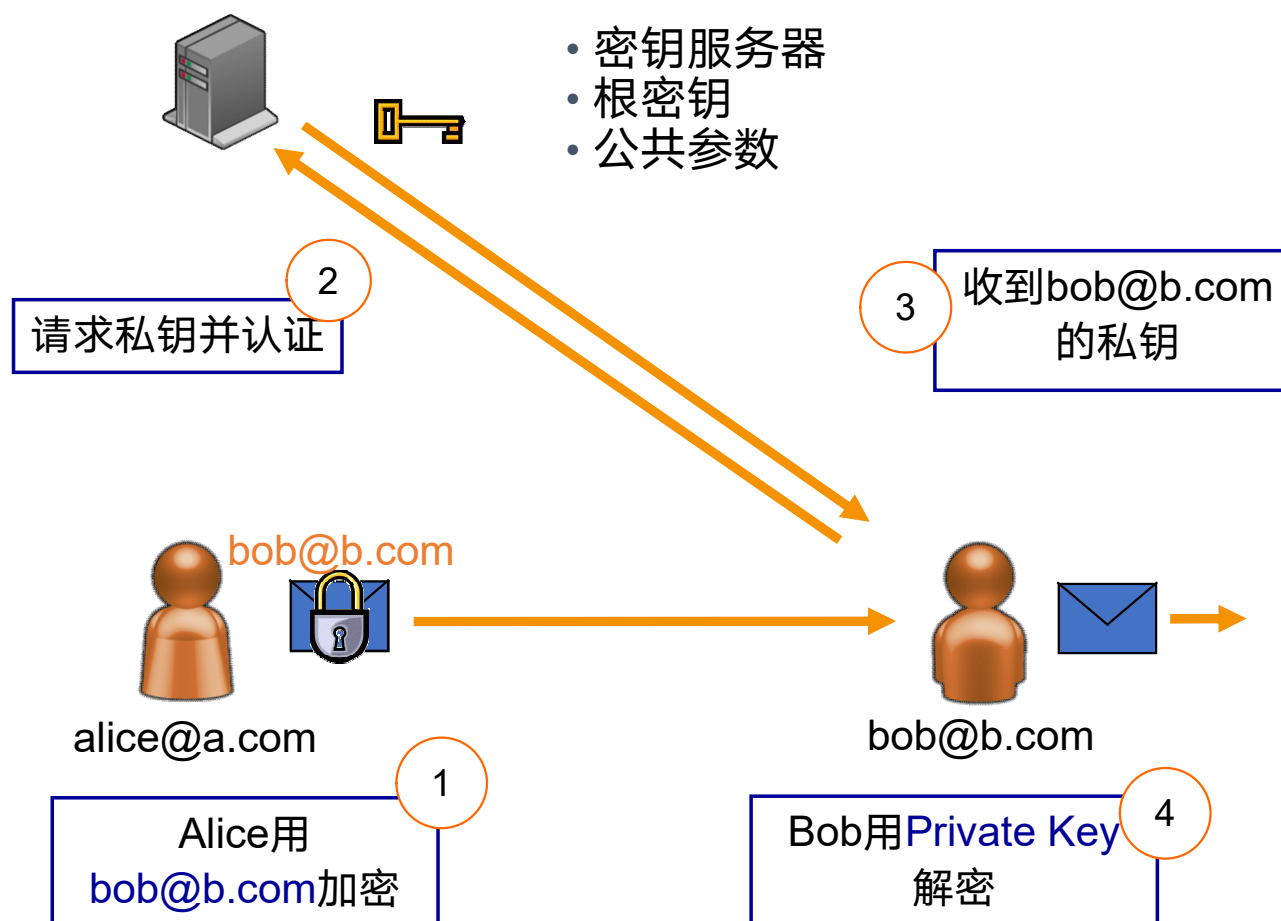


IBE介绍

6.1.2 实际的IBE

B U P T

□ 第一种情况是有密钥服务器的情况，以Alice发送信息给Bob为例：





IBE介绍

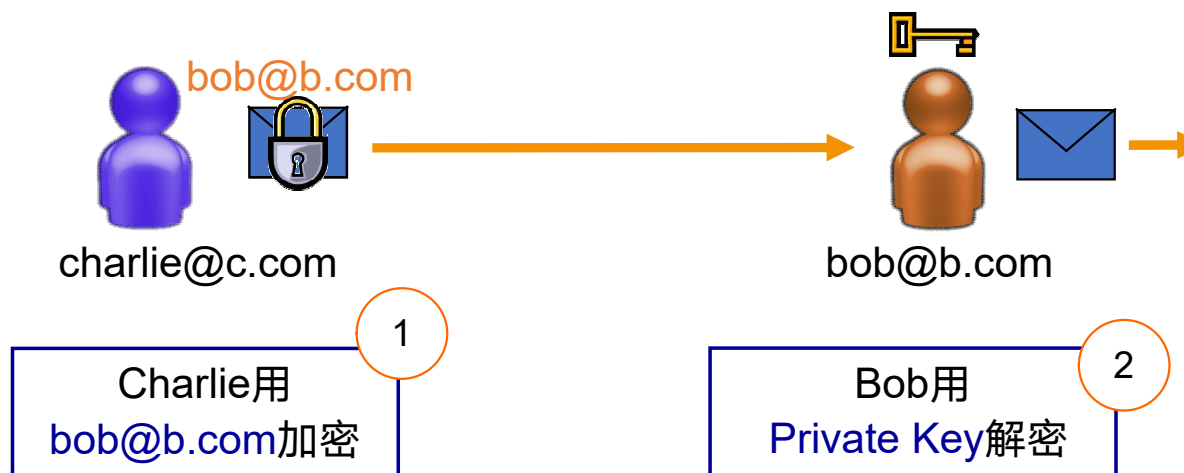
6.1.2 实际的IBE(2)

B U P T

□ 第二种情况是密钥服务器离线的情况，以Charlie发送信息给Bob为例：



完全离线 - 不需要连接服务器





□ 接下来以Voltage系统为例介绍IBE的应用实例。IBE的公钥组成如下：

IBE Public Key: **bob@wellsfargo.com** || **week = 252**

e-mail address

key validity

□ IBE系统使用短期密钥

- 公钥并非长期有效，它具有有效期
- 公钥每周都会发生变化，因此用户每周都必须获取新的私钥
- 密钥更新周期可以配置

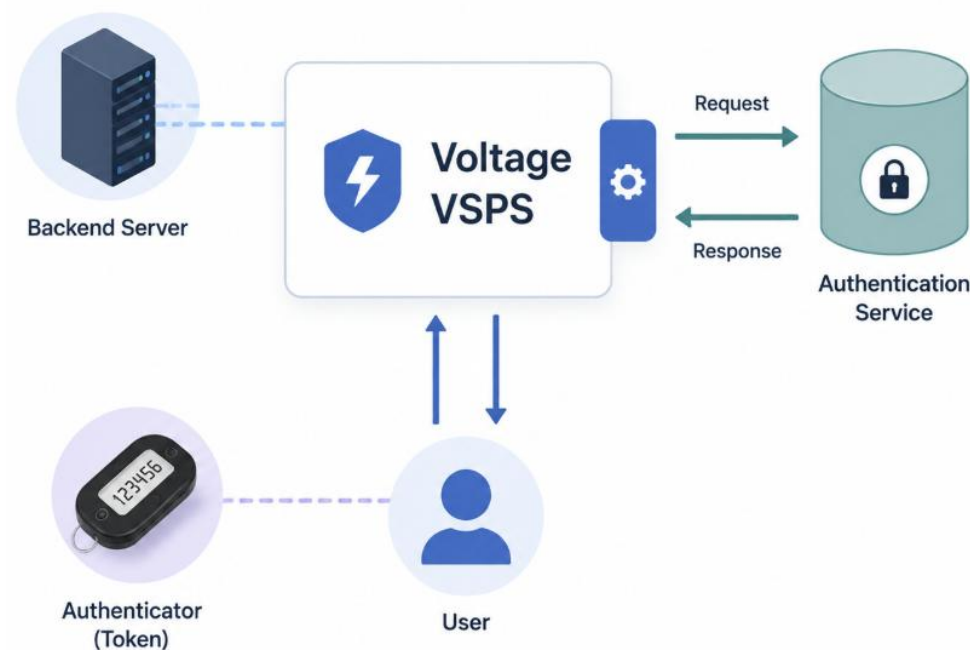


IBE介绍 6.1.3 用户认证

B U P T

□ Voltage(系统名称)能够支持任何类型的认证，身份验证需求因应用程序而异。因此，Voltage的IBE加密规模支持所有级别：

- PKI 智能卡
- RSA SecurID
- LDAP, Active Directory
- Login/Password
- Email Answerback
- 用户名和密码





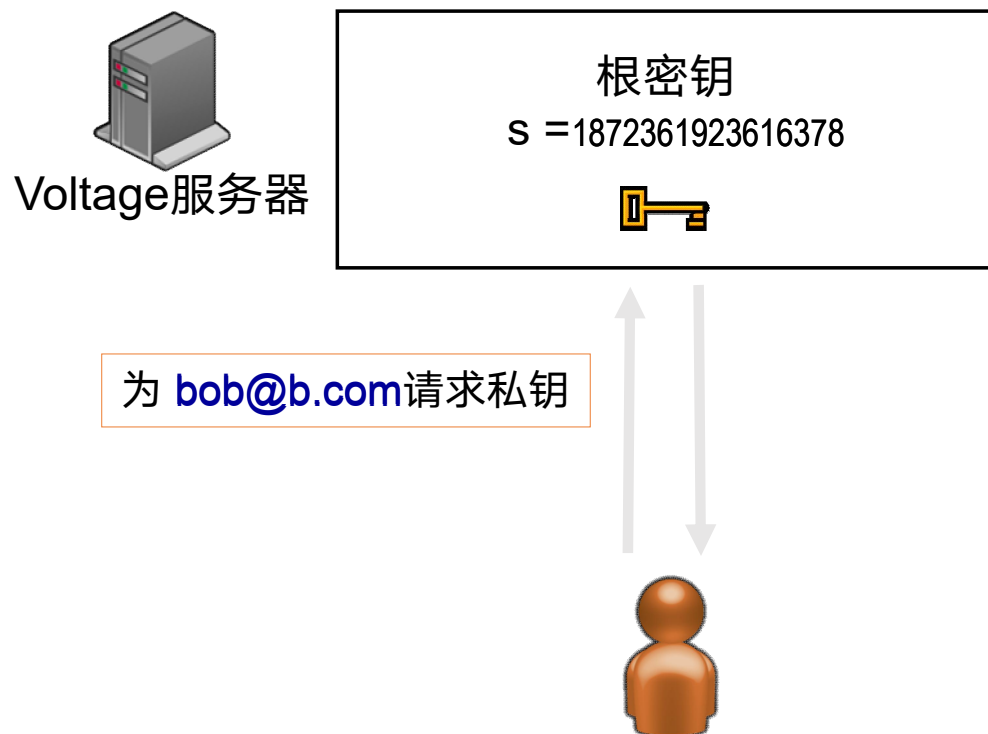
IBE介绍

6.1.3 IBE服务器

B U P T

□ 密钥服务器拥有用于生成密钥的根密钥

- 服务器设置时会随机选择一个根密钥
- 每个组织都有不同的根密钥
- 私钥由根密钥和身份信息生成





IBE介绍

6.1.3 IBE的操作

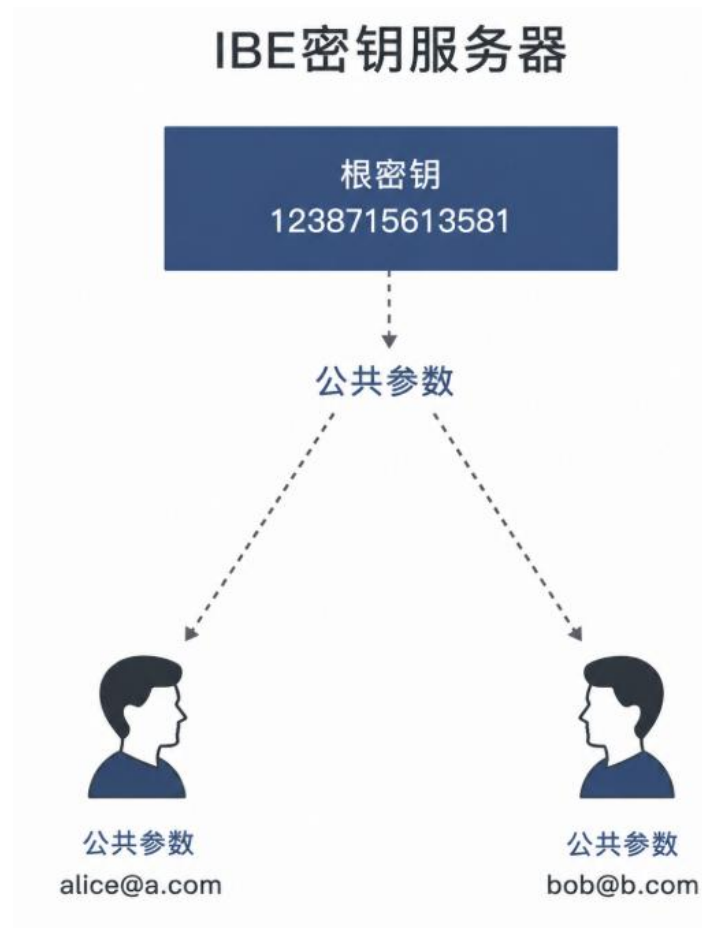
B U P T

□ 密钥服务器完成如下操作：

- 生成一个随机的根密钥
- 从根密钥推导出公共参数
- 向所有客户端分发公共参数(一次性设置)
- 公共参数类似于 CA 根证书（长期有效，与软件捆绑）。

□ 运行期间：

- 客户端在加密操作中使用公共参数
- 服务器使用根密钥为用户生成私钥



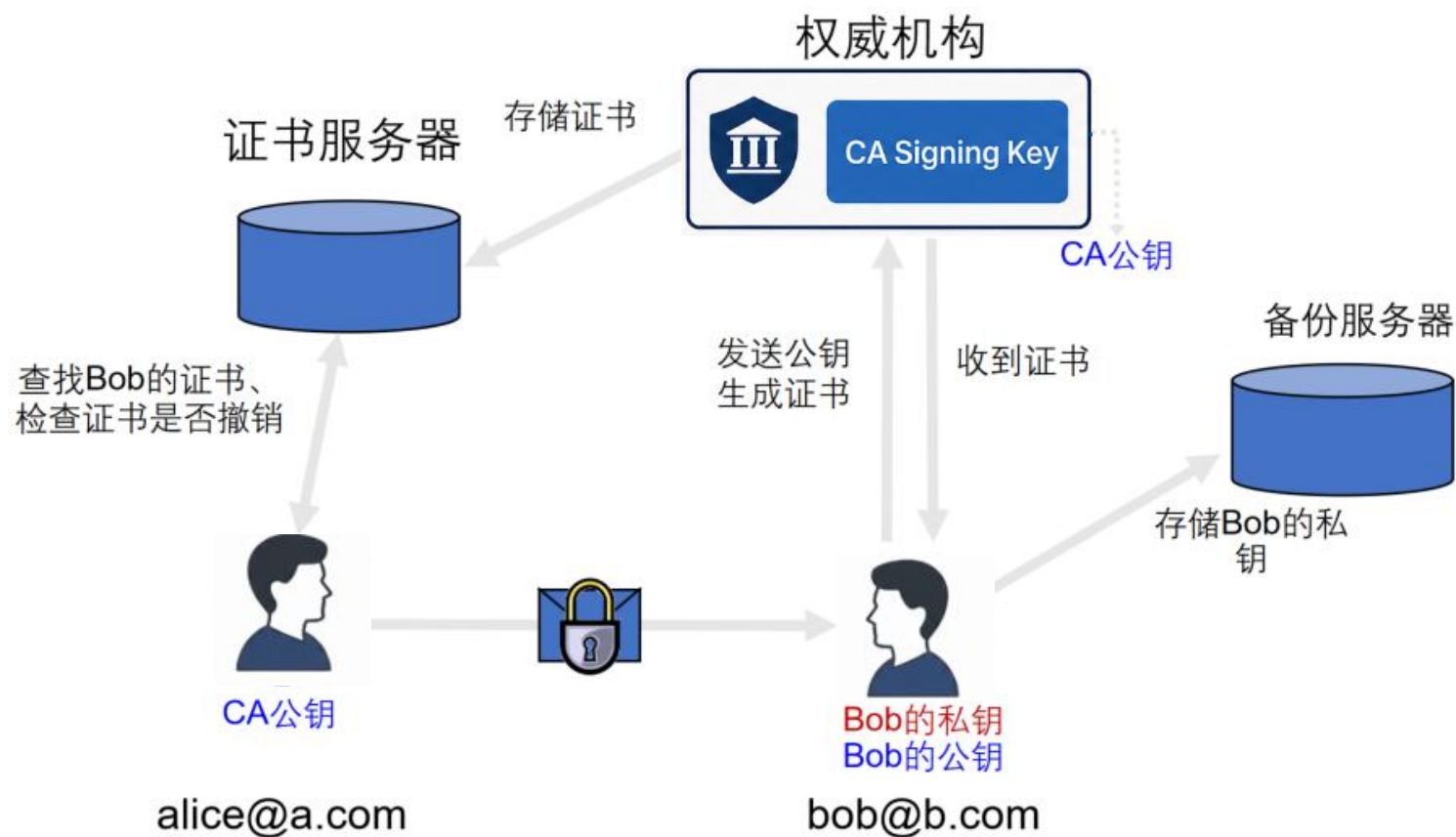


IBE介绍

6.1.4 IBE与PKI的对比

B U P T

□ PKI的证书服务器将身份与公开密钥绑定，如下所示：

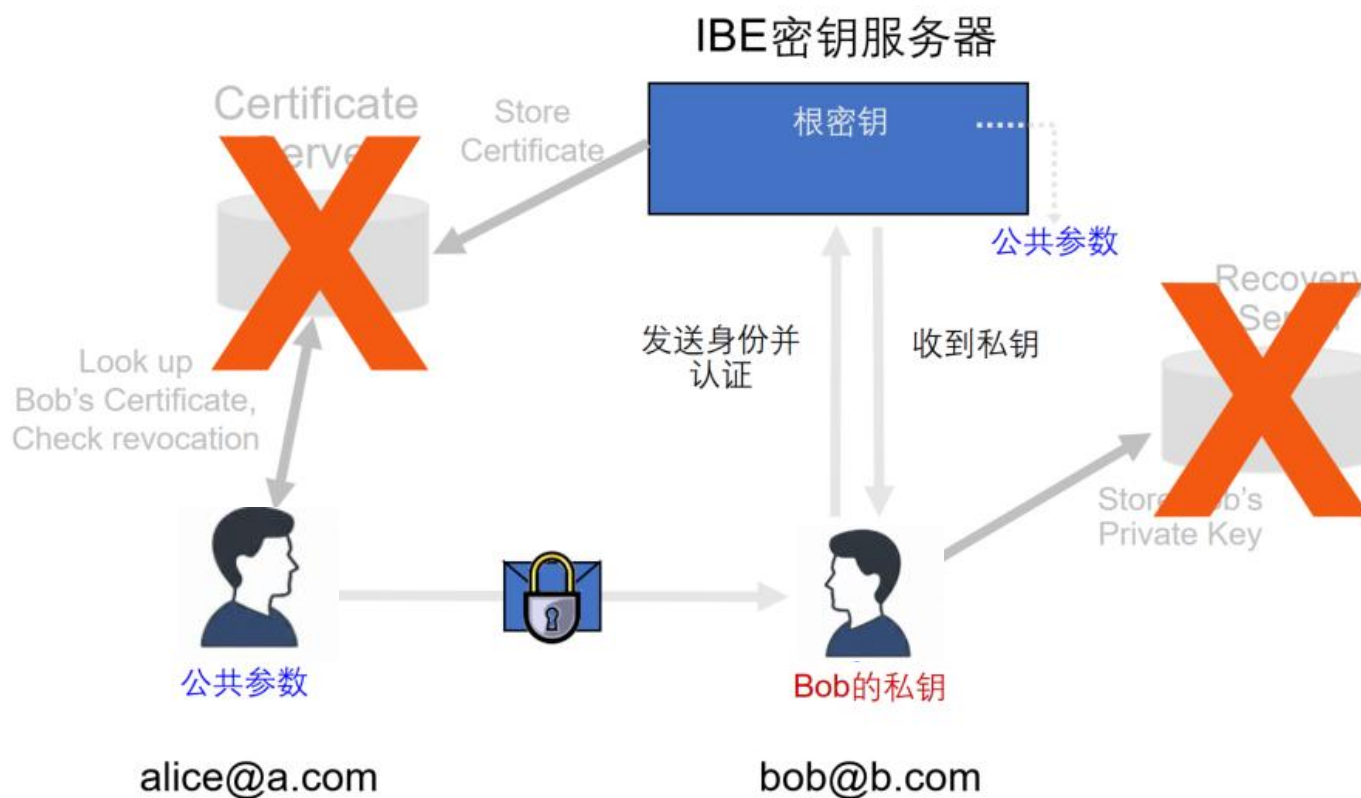




IBE介绍 6.1.4 IBE与PKI的对比(2)

B U P T

□ IBE的身份与密钥的绑定是隐式的，如下所示：





身份基加密的出现是为了解决公钥基础设施中的诸多问题的，下表展示了PKI中存在的问题以及IBE中是如何解决的。

问题	PKI 中的问题描述	IBE 如何解决
公钥分发	需要安全地分发公钥，确保公钥的真实性和完整性	无需预先分发公钥；密钥由身份标识直接导出
密钥管理复杂性	大量密钥需要维护，更新和撤销	减少了密钥的数量；密钥生成中心统一管理
证书管理	需要维护证书链，验证证书的有效性	没有证书管理的需求；减少了证书管理的负担
密钥撤销	密钥撤销后需要更新证书	密钥生成中心可直接撤销密钥，无需更新证书
性能瓶颈	在大规模部署时，证书的验证和管理成为性能瓶颈	减少了证书验证的需要，提高了效率
用户身份验证	需要额外机制来验证用户身份	身份标识作为加密和解密的一部分，内置了身份验证
密钥生命周期管理	密钥和证书的有效期管理复杂	简化了密钥生命周期管理，密钥可以根据需要生成和撤销



▣ IBE有着显著的优势，同时也存在一些局限性。下面用一个表格直观展示：

	描述	
优势	简化密钥管理	用户的身份标识（如电子邮件地址）直接用作公钥，无需预先分发公钥证书。
	增强互操作性	不需要依赖于特定的证书颁发机构（CA），IBE方案可以在不同的组织和系统之间实现更好的互操作性。
	提高安全性	IBE方案可以设计成在某些方面提供更强的安全保障，如紧密的安全归约和较低的运算量。
	适应移动和分布式环境	更适合移动设备和分布式系统，因为它们简化了密钥管理和证书管理。
不足	信任集中	IBE方案依赖于一个可信的密钥生成中心（KGC），这意味着如果KGC被攻破，整个系统的安全性将受到威胁。
	密钥撤销问题	密钥撤销在IBC方案中比在传统PKI中更加复杂，因为用户的私钥是由KGC派生的。
	性能开销	IBC方案中的某些操作可能比传统的公钥加密方案更耗时，尤其是在使用复杂的数学工具（如双线性配对）时。
	安全性证明的复杂性	IBC方案的安全性证明通常较为复杂，这增加了方案设计和分析的难度。



1. IBE有什么优点？存在什么问题？



6.2

国密SSL实现





- ✓ 6.2.1 SSL VPN协议
- ✓ 6.2.2 国密SSL浏览器实例
- ✓ 6.2.3 国密应用举例



- 记录层协议是SSL VPN中所有其它协议的底层协议
- 其它的所有协议都是封装在记录层协议中进行传输
- 功能：对传输数据进行分段、压缩、解压缩、加密、解密、完整性校验等操作
 - 针对被传输的数据：依次数据分段、压缩、计算HMAC、加密
 - 针对接收到的数据：依次解密、验证、解压缩、重新封装



□记录层协议将数据分成 2^{14} 字节或者更小的片段，每个片段使用如下结构体记录：

TLSPplaintext
ContentType type; ProtocolVersion version; Uint16 length; Opaque fragment[TLSPplaintext.length];

□type与version分别表示记录层协议类型与版本号

□length表示fragment片段长度

□fragment中为要传输的数据片段



- 记录层协议中的分块数据被指定的压缩算法进行压缩，压缩操作将TLSPlaintext结构数据转化为TLSCompressed结构数据

```
ContentType type;  
ProtocolVersion version;  
Uint16 length;  
Opaque fragment[TLSPlaintext.length];
```

- fragment为TLSPlaintext.fragment的压缩形式



- 通过加密和校验运算， TLSCompressed 结构的数据转化为 TLSCiphertext 结构数据

```
ContentType type;  
ProtocolVersion version;  
Uint16 length;  
Select (CipherSpec.cipher_type){  
    case block: GenericBlockCipher;  
} fragment;
```

- Fragment 是带校验码的 TLSCompressed.fragment 加密形式



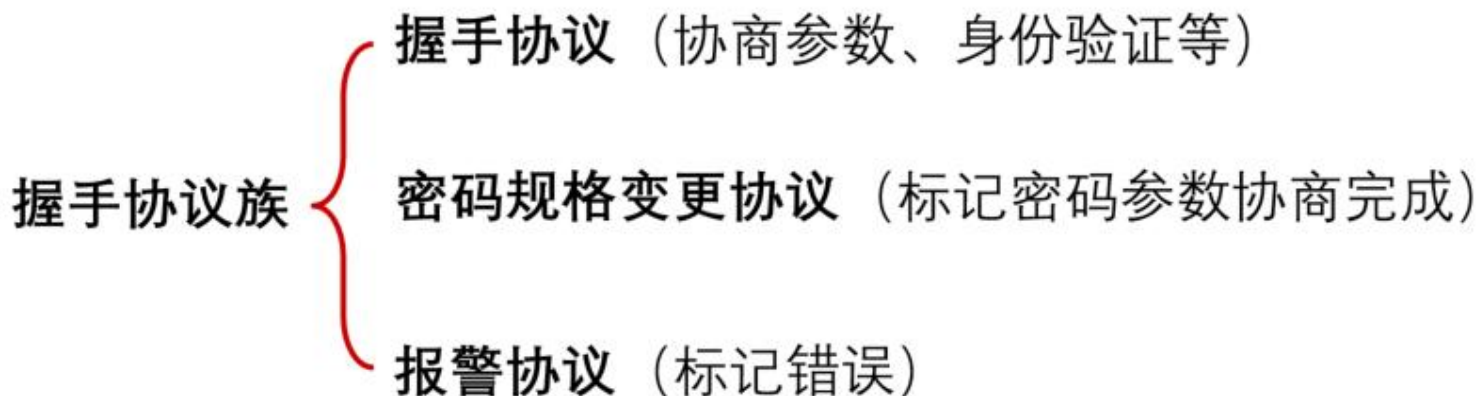
□ 校验码在加密之前进行计算，方式如下：

$$\begin{aligned} &MAC \\ &= HMAC_hash(MAC_{write_secret}, seq_num + TLSCompressed.type \\ &+ TLSCompressed.version + TLSCompressed.length \\ &+ TLSCompressed.fragment) \end{aligned}$$

□ 一般使用分组算法对数据进行加密，待加密的数据包含：
TLSCompress.fragment、MAC、padding_length、padding



- 握手协议族负责协商会话，此会话通常包括：会话标识、证书、压缩方法、密码规格、主密钥、重用标识





- 密码规格变更协议
- 功能：此协议用于通知密码规则的改变，初次协商时此消息为明文，否则会使用之前参数加密
- 格式：由一条消息组成，长度为一个字节，值为1
- 双方都需要在参数协商完毕后、握手消息结束之前发送此消息

ChangeCipherSpec
enum { change_cipher_spec(1),(255)} type;



□报警协议

□功能：用户关闭连接的通知以及对整个连接过程中出现的错误进行报警。报警协议存在两种情况：关闭通知和错误报警

□格式：由一条消息组成，长度为两个字节，分为报警级别和报警内容

关闭通知，发送方发送此通知后不再发送任何数据

错误报警，当发送或接受到一个致命级别的报警后，双方都因立即关闭连接



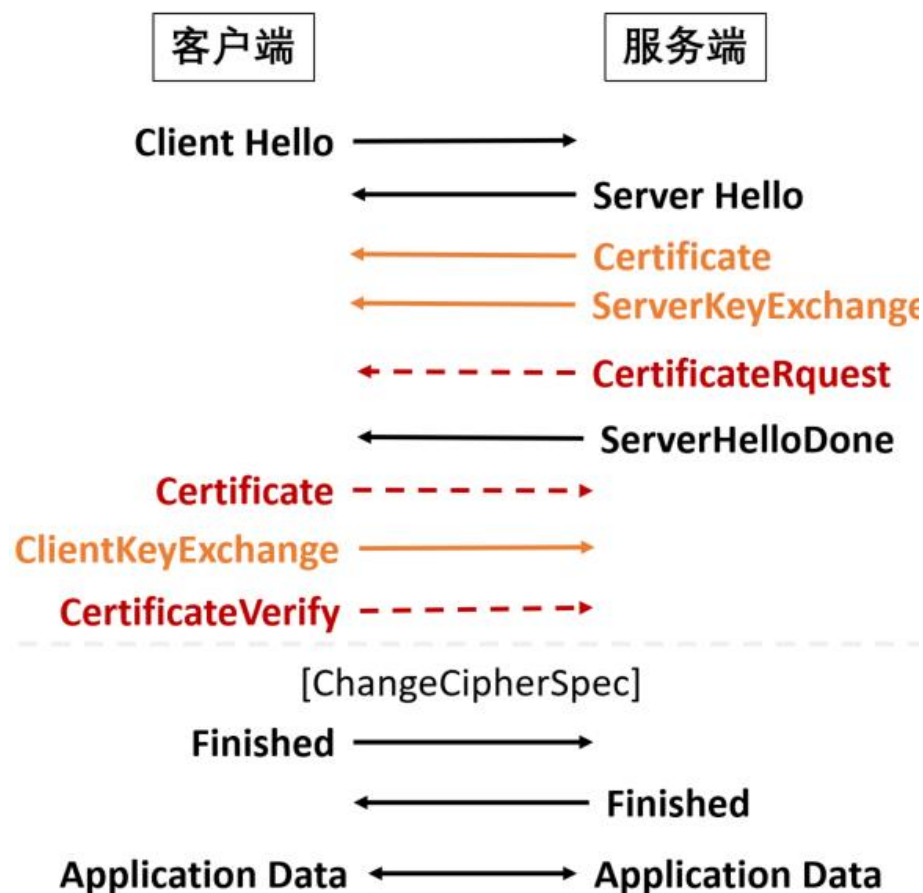
错误报警表

报警名称	值	级别	描述
Unexpected_message	10	致命	接收到一个不符合上下文关系的消息
Bad_record_mac	20	致命	MAC校验错误或解密错误
Decryption_failed	21	致命	解密失败
Record_overflow	22	致命	报文过长
Decompression_failure	30	致命	解压缩失败
Handshake_failure	40	致命	协商失败
Bad_certificate	42		证书被破坏
Unsupported_certificate	43		不支持的证书类型
Certificate_revoked	44		证书被撤销
Certificate_expired	45		证书过期或未生效
Certificate_unknown	46		未知证书错误
Illegal_parameter	47	致命	非法参数
Unknown_ca	48	致命	根证书不可信
Access_denied	49	致命	拒绝访问
Decode_error	50	致命	消息解码失败

报警名称	值	级别	描述
Decrypt_error	51		消息解密失败
Protocol_version	70	致命	版本不匹配
Insufficient_security	71	致命	安全性不足
Internal_error	80	致命	内部错误
User_canceled	90	警告	用户操作错误
Unsupported_site2site	200	致命	不支持site2site
No_area	201		没有保护域
Unsupported_areatype	202		不支持的保护域
Bad_ibcparam	203	致命	接收到一个无效的ibc公共参数
Unsupported_ibcparam	204	致命	不支持ibc公共参数中定义的信息
Identity_need	205	致命	缺少对方的ibc标识



- Step1: 客户端发送hello消息
- Step2: 服务端回复 hello, 并发送证书与密钥交换消息, 可选择向客户端索要证书。
- Step3: 客户端回复密钥交换消息, 如果服务端索要了证书, 还会发送证书与证书验证消息
- Step4: 客户端与服务端用已协商的密钥加密接下来的Finish消息与传输数据





国密SSL实现

6.2.1 握手协议族(6)

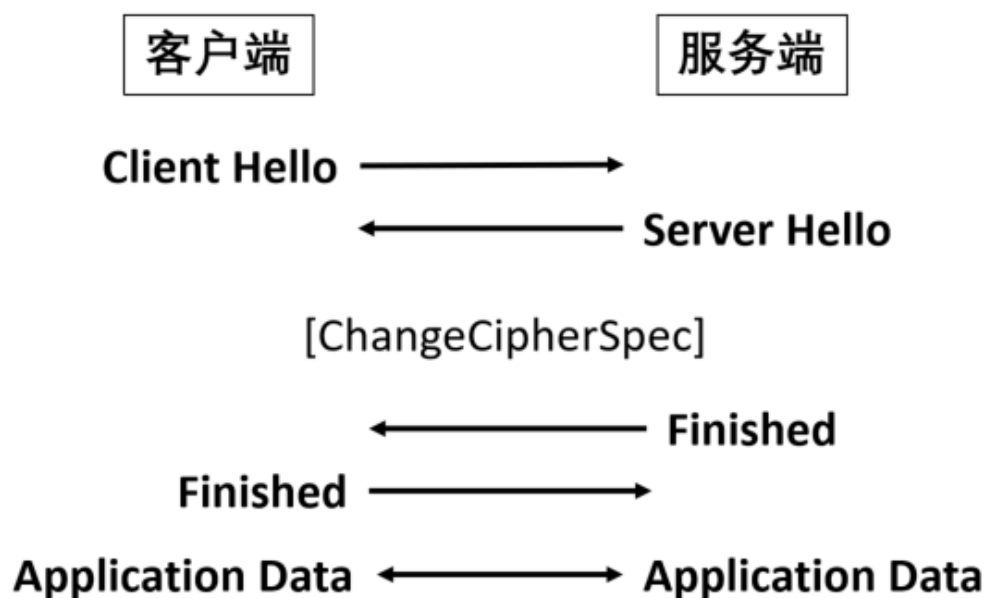
B U P T

□重用握手协议：双方可以通过设置重用会话标识来重用之前的会话而不必重新协商安全参数。

□Step1：客户端的Client Hello中带上了重用会话标识。

□Step2：服务端若有相匹配的会话存在则使用此会话状态接受连接。

□Step3：双方发送密码规格变更消息以及握手结束消息后开始进行数据传输。





□ Hello消息为握手协议的第一条消息，其中ClientHello消息结构如下：

```
ProtocolVersion version;  
Random random;  
SessionID session_id;  
CipherSuite cipher_suites;  
CompressionMethod compression_methods;
```

□ cipher_suites表示客户端所支持的密码套件列表

□ compression_methods表示客户端所支持的压缩函数列表



- 在Hello阶段，客户端与服务器都会向对方发送支持的密码套件列表
- 单个密码套件包含：一个密钥交换算法、一个加密算法和一个校验算法
- 协议中可选的算法有：
 - 秘钥交换算法：ECDHE/ECC（SM2）、IBSDH/IBC（SM9）、RSA
 - 加密算法：SM1、SM4
 - 校验算法：SM3、SHA1



序号	名称	值
1	ECDHE_SM1_SM3	{0xe0,0x01}
2	ECC_SM1_SM3	{0xe0,0x03}
3	IBSDH_SM1_SM3	{0xe0,0x05}
4	IBC_SM1_SM3	{0xe0,0x07}
5	RSA_SM1_SM3	{0xe0,0x09}
6	RSA_SM1_SHA1	{0xe0,0x0a}
7	ECDHE_SM4_SM3	{0xe0,0x11}
8	ECC_SM4_SM3	{0xe0,0x13}
9	IBSDH_SM4_SM3	{0xe0,0x15}
10	IBC_SM4_SM3	{0xe0,0x17}
11	RSA_SM3_SM3	{0xe0,0x19}
12	RSA_SM4_SHA1	{0xe0,0x1a}



- 证书消息紧跟在Hello消息之后，依据不同的密码套件传送不同的整数或公共参数，证书结构如下：

Certificate(IBC)
opaque ibc_id; ASN.1 IBCParam ibc_parameter;

Certificate(Cert)
ASN.1Cert certificate;



下面表格展示了不同的KeyExchange算法与对应的Certificate

密钥交换算法	Certificate	Certificate Key
RSA	签名证书与加密证书	RSA公钥
ECC/ECDHE (SM2)	签名证书与加密证书	ECC公钥
IBC/IBSDH (SM9)	唯一标识与IBC公共参数	



- 服务端的密钥交换消息用于给客户端计算生成预主密钥，依照选择的密码套件的不同，有不同的结构

ECDHE(SM2)	ECC
ServerECDHEParam params; Digitally-signed struct{ opaque client_random; opaque server_random; ServerECDHEParams params; }signed_params;	Digitally-signed struct{ opaque client_random; opaque server_random; opaque ASN.1 Cert; }signed_params;



IBSDH(SM9)

```
ServerIBSDHParam params;  
Digitally-signed struct{  
    opaque client_random;  
    opaque server_random;  
    ServerIBSDHParams params;  
}signed_params;
```

RSA

```
Digitally-signed struct{  
    opaque client_random;  
    opaque server_random;  
    opaque ASN.1 Cert;  
}signed_params;
```

IBC

```
ServerIBCPParam params;  
Digitally-signed struct{  
    opaque client_random;  
    opaque server_random;  
    ServerIBCPParams params;  
    opaque IBCEncryptionKey;  
}signed_params;
```



□ 网关到网关协议定义了：

- SSL VPN之间建立网关到网关的传输层隧道
- 对IP数据报文进行安全传输时所采用的报文格式（包括控制报文与数据报文）
- 控制报文交换过程和数据报文封装过程

□ 保护域：允许建立连接的IP地址范围以及端口范围

□ 数据报文管理：网关之间通过握手协议建立SSL连接，并将此连接与本地和对端的保护域进行绑定，基于此保护域管理出入站报文

□ SSL VPN产品中，客户端-服务端模式为必备模式，而网关-网关模式为可选模式



□ 网关到网关协议包含两种协议报文：控制报文、数据报文。结构如下：

Site2Site	Site2SiteCmd
<pre>Site2SiteContentType site2sitecontenttype; Uint16 length; Select (site2sitecontenttype){ case command Site2SiteCmd; case data: Site2SiteData; } Site2Sitefragment;</pre>	<pre>enum {cmd_area(0),(255)} CmdType; Struct{ CmdType cmdtype; select(cmdtype){ case cmd_area: Arealist arealist; } } CmdFragment;</pre>

□ Site2SiteCmd为控制报文，定义了保护域

□ Site2SiteData为数据报文，内容为原始的完整IP报文



□ 控制报文交换：

- 使用握手协议协商好的安全参数进行保护
- 在握手协议完成之后的任何时间都可发送
- 由通讯双方任何一方发起
- 通知对方自己所保护的网路信息

□ 数据报文出入站：

- 建立SSL连接，将连接与本地和对端的保护域进行绑定
- SSL VPN依据报文的源IP和目的IP查找对应匹配的保护域，并获取相应的有效SSL连接
- 将IP报文作为记录层的数据内容封装在此匹配连接的记录层上



□实验环境：

- <https://www.gmssl.cn/gmssl/>
- 支持国密的浏览器（推荐360浏览器/奇安信浏览器）
- 本地web服务器（Tomcat）
- Java环境（国密算法以jar包形式集成入Tomcat）
- 本课件演示的实验中：
- 360浏览器版本：13.1.1302.0
- Java版本：jdk8u821
- Tomcat版本：9.0.45



- 需 要 从 <https://www.gmssl.cn/gmssl/> 中 下 载 gmjce.jar, gmjsse.jar 与 gmssl4t.jar, 并创建国密证书sm2.pfx文件
- 将gmjce.jar与gmjsse.jar放入java安装路径中的 “jre/ext” 目录下
- 将gmssl4t.jar放入Tomcat安装路径的 “lib”目录下
- 设置Tomcat的 “conf/server.xml” 文件中的服务器信息如下:

```
<Connector port="443"  
.....protocol="HTTP/1.1"  
.....SSLEnabled="true"  
.....sslImplementationName="cn.gmssl.tomcat.GMSSLImplementation"  
.....sslProtocol="GMSSLv1.1"  
.....keystoreFile="D:\Tomcat\sm2.Lsy\sm2.Lsy.both.pfx"  
.....keystoreType="PKCS12"  
.....keystorePass="12345678">  
</Connector>
```

服务器端口

创建的国密证书路径



在360安全浏览器中设置支持国密SSL协议



将浏览器设置为极速模式，因为兼容模式使用IE内核，不支持国密。



国密SSL实现

6.2.2 国密服务器演示

B U P T

- 开启 Tomcat 的 web 服务器后，在浏览器中输入本机 ip 与端口号（例如 `https://192.168.2.158:443`），成功访问网站





- 本机服务器与本机传输数据默认不经过网卡因此无法使用WireShark抓包，使用以下命令设置本机数据包传输路由使其强制经过网卡：

```
C:\Windows\system32>route add 192.168.2.158 mask 255.255.255.255 192.168.2.1  
操作完成!
```

本机ip地址

网关地址

注：需要使用管理员权限打开命令行



□ 57679为客户端端口，443为服务器端口

Source Port	Destination Port	Info	
57679	443	Client Hello	Client Hello
57679	443	57679 → 443 [ACK] Seq=1 Ack=1 Win=65700 Len=0	
443	57679	Server Hello, Certificate, Server Key Exchange, Server Hello Done	Server Hello, Certificate, ServerKeyExchange, ServerHelloDone
57679	443	57679 → 443 [ACK] Seq=105 Ack=2059 Win=65700 Len=0	
57679	443	Client Key Exchange, Change Cipher Spec, Encrypted Handshake Message	ClientKeyExchange, ChangeCipherSpec, Finished
443	57678	Alert (Level: Warning, Description: Close Notify)	
443	57678	443 → 57678 [FIN, ACK] Seq=2066 Ack=113 Win=65536 Len=0	
57678	443	57678 → 443 [RST, ACK] Seq=113 Ack=2066 Win=0 Len=0	
57678	443	57678 → 443 [RST, ACK] Seq=113 Ack=2066 Win=0 Len=0	
443	57677	Alert (Level: Warning, Description: Close Notify)	
443	57677	443 → 57677 [FIN, ACK] Seq=2067 Ack=113 Win=65536 Len=0	
57677	443	57677 → 443 [RST, ACK] Seq=113 Ack=2067 Win=0 Len=0	
57677	443	57677 → 443 [RST, ACK] Seq=113 Ack=2067 Win=0 Len=0	
443	57679	Change Cipher Spec	ChangeCipherSpec, Finished
443	57679	Encrypted Handshake Message	
57679	443	57679 → 443 [ACK] Seq=363 Ack=2150 Win=65608 Len=0	
57679	443	Application Data	Application Data
443	57679	Application Data	
57679	443	Application Data	

□ 搭建服务器时设置为单向验证，所以服务器没有向客户端请求证书



Client握手包格式如下

记录层

握手协议

- GMSSLv1 Record Layer: Handshake Protocol: Client Hello
 - Content Type: Handshake (22)
 - Version: GMSSL 1.0 (0x0101)
 - Length: 67
 - Handshake Protocol: Client Hello
 - Handshake Type: Client Hello (1)
 - Length: 63
 - Version: GMSSL 1.0 (0x0101)
 - Random: 0d8a15019a08f6daad2339c785d5845b4b15905d65be73f1...
 - Session ID Length: 0
 - Cipher Suites Length: 4
 - Cipher Suites (2 suites)
 - Compression Methods Length: 1
 - Compression Methods (1 method)
 - Extensions Length: 18
 - Extension: application_layer_protocol_negotiation (len=14)

Client_version:协议版本

Cipher_suites:密码套件列表

Compression_methods:压缩算法



Server握手包格式如下

```
✦ GMSSLv1 Record Layer: Handshake Protocol: Multiple Handshake Messages
  Content Type: Handshake (22)
  Version: GMSSL 1.0 (0x0101)
  Length: 2053
```

```
✦ Handshake Protocol: Server Hello
  Handshake Type: Server Hello (2)
  Length: 70
  Version: GMSSL 1.0 (0x0101)
  Random: 609e2065adc124b671268b6dbf3ddae7fd89bc03bc6394f8...
  Session ID Length: 32
  Session ID: 609e2065ce41d8b294c711ec22c61f8f99a4c0339f61dbcb...
  Cipher Suite: ECC_SM4_SM3 (0xe013)
  Compression Method: null (0)
```

```
✦ Handshake Protocol: Certificate
  Handshake Type: Certificate (11)
  Length: 1894
  Certificates Length: 1891
  Certificates (1891 bytes)
```

```
✦ Handshake Protocol: Server Key Exchange
  Handshake Type: Server Key Exchange (12)
  Length: 73
```

```
✦ Handshake Protocol: Server Hello Done
  Handshake Type: Server Hello Done (14)
  Length: 0
```

四个握手消息封装在一个记录层协议中

ServerHello与ClientHello格式相同

服务器证书

密钥交换消息（用于产生预主密钥）

握手结束消息



Client接受到服务器证书后的响应

- GMSSLv1 Record Layer: Handshake Protocol: Client Key Exchange
 - Content Type: Handshake (22)
 - Version: GMSSL 1.0 (0x0101)
 - Length: 161
 - Handshake Protocol: Client Key Exchange
 - Handshake Type: Client Key Exchange (16)
 - Length: 157
- GMSSLv1 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
 - Content Type: Change Cipher Spec (20)
 - Version: GMSSL 1.0 (0x0101)
 - Length: 1
 - Change Cipher Spec Message
- GMSSLv1 Record Layer: Handshake Protocol: Encrypted Handshake Message
 - Content Type: Handshake (22)
 - Version: GMSSL 1.0 (0x0101)
 - Length: 80
 - Handshake Protocol: Encrypted Handshake Message

三个记录层协议

密钥交换消息（包含使用服务器公钥加密后的预主密钥）

Client密码规格变更协议

Client Finished消息，使用新密钥加密



□ Server针对密钥规格变更的响应

```
└─ GMSSLv1 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
  Content Type: Change Cipher Spec (20)
  Version: GMSSL 1.0 (0x0101)
  Length: 1
  Change Cipher Spec Message
```

Server密码规格变更协议

```
└─ GMSSLv1 Record Layer: Handshake Protocol: Encrypted Handshake Message
  Content Type: Handshake (22)
  Version: GMSSL 1.0 (0x0101)
  Length: 80
  Handshake Protocol: Encrypted Handshake Message
```

Server Finished, 使用新
密钥加密



□ Server针对密钥规格变更的响应

```
└─ GMSSLv1 Record Layer: Change Cipher Spec Protocol: Change Cipher Spec
  Content Type: Change Cipher Spec (20)
  Version: GMSSL 1.0 (0x0101)
  Length: 1
  Change Cipher Spec Message
```

Server密码规格变更协议

```
└─ GMSSLv1 Record Layer: Handshake Protocol: Encrypted Handshake Message
  Content Type: Handshake (22)
  Version: GMSSL 1.0 (0x0101)
  Length: 80
  Handshake Protocol: Encrypted Handshake Message
```

Server Finished, 使用新
密钥加密



- 随着国密算法的推广延伸，金融领域引入SM2、SM3、SM4等算法逐步替代原有的RSA、ECC等国外算法，现有的银联银行卡联网、银联IC两项规范都引入了国密算法的相关要求

网上证券和基金	身份认证、资用户信息查询、委托交易、转账操作、行情和资讯处理
网上开户	数字签名、身份认证、银证绑定、资料对接
证券商系统	用户密码管理、交易和账户系统管理、软硬件设备安全



- CAN总线是目前使用最广泛的车载总线网络技术，一旦攻击者通过外部访问接口渗透到连接关键控制单元的CAN总线网络并发送恶意报文，则会严重影响汽车运行状况
- 可以将国密SM4集成至微控制器中，在传输CAN总线数据时先进行数据加密，可有效提高车联网安全性

参考文献：陈刚.国密SM4算法在车载CAN总线的加密应用[J].信息通信,2019(03):149-151.



- 目前智能变电站系统安全防护主要依据IEC 62351标准实施，IEC 62351 是国际标准，其涉及的安全技术主要采用国际算法RSA、AES及数字证书。
- 可以采用国密SM2算法和调度数字证书实现调度主站和远动机之间的双向身份认证。
- 使用基于国密SM2、SM3、和SM4 算法及调度数字证书建立TLS 通信协议确保调度主站与变电站之间的安全报文传输。



2.国密SSL在工业领域还有什么应用?

谢谢大家
